Builder cleverly bends the syntax of Ruby to support nested tags, attributes, and other niceties. The core idea of Builder is simple: calls such as name and language are processed by XmlMarkup#method_missing, which generates an XML tag for every call.

Now pretend you have to generate a piece of XML describing a university course. It might look like this:

```
<semester>
  <class>Egyptology</class>
  <class>Ornithology</class>
</semester>
```

So, you'd have to write code like this:

methods/builder_example_2.rb

```
xml.semester {
  xml.class 'Egyptology'
  xml.class 'Ornithology'
}
```

If XmlMarkup were a subclass of Object, then the calls to class would clash with Object's class. To avoid that clash, XmlMarkup inherits from a *Blank Slate (66)* that removes class and most other methods from Object. When Builder was written, BasicObject didn't exist yet. (It was introduced in Ruby 1.9.) So Builder defines its own Blank Slate class:

gems/builder-3.2.2/lib/blankslate.rb

```
class BlankSlate
  # Hide the method named +name+ in the BlankSlate class.  Don't
  # hide +instance_eval+ or any method beginning with "__".
  def self.hide(name)
    # ...
    if instance_methods.include?(name._blankslate_as_name) &&
        name !~ /^(__|instance_eval$)/
      undef_method name
    end
  end
  # ...

  instance_methods.each { |m| hide(m) }
end
```

Builder doesn't go as far as removing each and every method from BlankSlate. It keeps instance_eval (a method that you'll get to know in the next chapter) and all the "reserved methods"—methods that are used internally by Ruby, whose names conventionally begin with a double underscore. One example of a reserved method is BasicObject#__send__, which behaves the same as send but gives you a scary warning when you try to remove it. The case of instance_eval